Autonomous Volcanic Terrain Exploration Using a Markov Decision Process

Project Update Report (Update 1)

Shakil Ahmed, Fahim Foysal, Shefa Tabassum, and Tanvir Ahmed
Group 5, CSE 440 — Artificial Intelligence, Section 1
North South University · Instructor: Dr. Mohammad Shifat-E-Rabbi

Abstract—This update reports the state of our semester project at the midpoint. The project models autonomous exploration of hazardous volcanic terrain as a Markov Decision Process (MDP). As of this update, all four members' Update-1 work is merged: a seeded procedural terrain generator, the complete MDP formulation with value iteration and policy extraction, a policy-following explorer agent with a simulation loop, matplotlib terrain visualization, and a three-agent baseline experiment framework with CSV and plot outputs. We summarize the design, show preliminary results, and state the plan to the final submission.

I. Problem and Approach

Volcanic environments are dangerous and uncertain: lava flows, craters, gas emissions, and impassable rock threaten any explorer, and a rover's movements on loose terrain are unreliable. Our project asks an autonomous agent, starting from a base station on a grid map, to collect science samples while avoiding hazards and surviving.

Because the task is sequential decision-making under uncertainty, we model it as an MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ and solve it exactly with value iteration:

$$V(s) \leftarrow \max_{a} \sum_{s'} P(s' \mid s, a)\,[R(s, a, s') + \gamma V(s')]\,. \qquad (1)$$

States are walkable grid cells; actions are the four movement directions plus stay and scan; movement succeeds as intended with probability 0.75, drifts sideways with probability 0.10 each, and stalls with probability 0.05; rewards range from +20 (science) to −100 (lava, fatal); and $\gamma = 0.90$.

II. Progress by Member

1) Member 1 — terrain and MDP core: Repository structure and configuration system; a seeded procedural terrain generator over seven cell types with hazard density capped at 0.30 and CSV save/load; the full MDP formulation; value iteration and optimal policy extraction.

2) Member 2 — agent and simulation: An MDPExplorerAgent that starts at the base station, queries the policy, and samples real outcomes from the transition model; tracking of reward, hazards entered, science collected, and survival; a simulation loop with a step budget and coverage target and a mission summary.

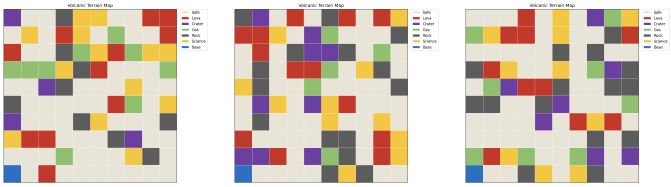

Fig. 1. Generated volcanic terrains (seeds 1, 7, 23).

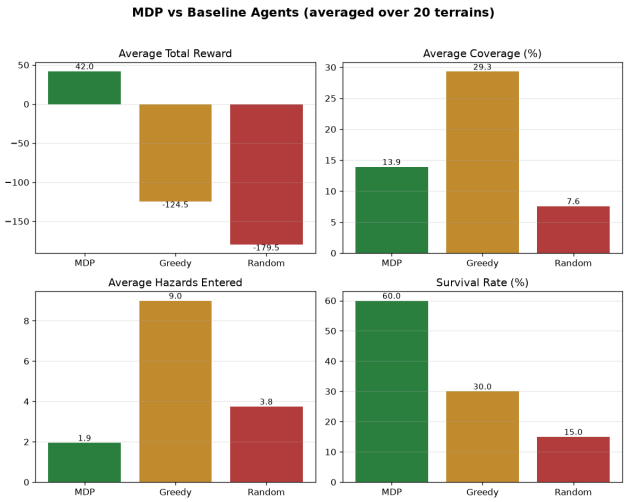

Fig. 2. Preliminary baseline comparison from the experiment framework.

3) Member 3 — visualization: Matplotlib rendering of the terrain grid with a distinct color per cell type, title, and legend (Fig. 1); example maps generated across several seeds; a layout-test exploration map (clearly labeled as a sample path, to be replaced by the real agent path).

4) Member 4 — experiments: A comparison harness running three agents on identical terrains: the MDP agent, a greedy nearest-unvisited baseline, and a random-walk baseline; shared metrics (reward, coverage, hazards, science, survival, steps); automatic CSV and performance-plot generation (Fig. 2).

III. Preliminary Observations

The value-iteration policy already routes visibly around lava and craters and toward science points, and the MDP agent outperforms both baselines on reward and survival in the preliminary runs. One issue is identified and scheduled for the next phase: because a science cell pays its reward on every entry, the optimal policy can "camp" on a single science cell; we will fix this with a collect-once rule and on-line re-planning.

IV. Plan to the Final Submission

1) Move the collect-once science rule with re-planning into the core agent (fixing the camping behavior everywhere).
2) Dynamic hazards: drifting gas clouds and spreading, cooling lava flows, with the agent re-planning after every terrain change.
3) Render the real simulated agent path on the mission map, replacing the sample path.
4) Upgrade main.py to run the full pipeline end to end.
5) Final report (IEEE format), presentation, and the one-minute demo video.

All code is on the group's public GitHub repository, developed through per-member branches and reviewed pull requests, following the mandated layout (main.py, README.md, requirements.txt, data/, support/, others/).